Reti Logiche

Andrea Salvatore e **Elisa De Salvo**

Anno accademico: 2026/2027

I miei contatti:

- **GitHub**: andrea-salvatore
- **Website**: andreasalvatore.dev
- **Cellulare**: +39 3703116443
- **E-mail**:
  - Personale: asalvatore.junior@gmail.com
  - Lavorativa: ajs.sdk.mt@gmail.com
  - Istituzionale: a.salvatore9@studenti.unipi.it

Contatti di Elisa De Salvo:

- **Cellulare**: +39 3703714218
- **E-mail**: desalvo.elisa@outlook.it

L'intento di questo file è contenere le lezioni di Reti Logiche del prof. Giovanni Stea, con il dott. Zippo come assistente di laboratorio, in ordine strettamente cronologico seguendo, dunque, le lezioni dell'anno accademico citato sopra. L'idea nasce dal fatto che spesso capita che gli assenti abbiano bisogno di recuperare delle lezioni. Attraverso i file in questo repository ora hanno tutto in tempo reale lezione dopo lezione. Inoltre, per senso di collaborazione, agli studenti è fornito non solo il PDF, bensì anche il sorgente .tex, utile per correggere errori e/o aggiungere dettagli utili direttamente tramite le Pull Request di GitHub. Oppure, nell'ipotesi che io sia in difficoltà con la trascrizione corretta degli appunti, ci si può aiutare inserendo tutto qui e mettendo a disposizione di **tutti** il materiale in ordine cronologico e in tempi ristretti. Grazie a tutti per la collaborazione.

Questa lezione, come le sue precedenti e successive, può contenere errori e/o imprecisioni. Per il bene di tutti, è buona norma far notare all'autore l'errore o l'imprecisione così da risolvere nel minor tempo possibile. I canali di comunicazione, per eventuali avvisi generali o di errori trovati negli appunti, sono in cima al file. Grazie per la collaborazione e Buono studio!

Lezione 1 | 22 settembre 2026

Introduzione: dalle reti logiche al linguaggio macchina, Assembly e assemblatore, La ALU, Struttura di un calcolatore

Introduzione: dalle reti logiche al linguaggio macchina

Reti logiche: si parla di hardware da una prospettiva **funzionale**. Ad esempio, una porta NOR esegue la stessa funzione indipendentemente da come è realizzata internamente.

A questo punto smettiamo di ragionare ad alto livello e iniziamo a ragionare a basso livello.

Un processore **preleva** un'istruzione dalla memoria e poi la **elabora**.

Integrazione

Questo è il ciclo *fetch–decode–execute*: prelievo (fetch) dell'istruzione dalla memoria, decodifica, esecuzione. Il ciclo si ripete per ogni istruzione.

Assembly e assemblatore

Come traduco gli 0 e gli 1 (linguaggio macchina) che ci sono nei registri e in memoria? Attraverso l'**Assembly**, che usa codifiche testuali (ASCII) per essere *human friendly*: si scrivono direttamente le **istruzioni da far eseguire al processore**.

- Il **linguaggio** si chiama **Assembly** (per noi "Assembler").
- Il programma che si occupa dell'**assemblaggio**, cioè della traduzione in linguaggio macchina, è l'**assemblatore**. Si tratta di una situazione molto diversa dalla compilazione: la traduzione è sostanzialmente uno-a-uno, un'istruzione Assembly corrisponde a un'istruzione macchina.

Nota: controllo di flusso

In Assembly mancano i **costrutti di controllo di flusso** dei linguaggi ad alto livello (`if`, cicli). Esiste l'istruzione `JMP`, che salta a un'altra parte del codice.

Integrazione

Oltre al salto incondizionato `JMP` esistono i **salti condizionali** (`JE`, `JNE`, `JL`, ...), che saltano in base ai flag impostati dall'ultima operazione della ALU. Combinando salti e confronti si realizzano `if` e cicli.

Caratteristiche

- Un processore manipola **stringhe di bit**. La loro **rappresentazione** (numero, carattere, indirizzo, ...) la conosciamo **noi e solo noi**: non ha nulla a che vedere con il linguaggio, e c'è poco aiuto da parte del sistema sottostante (nessun controllo sui tipi).
- L'Assembly è **specifico di un processore**: il codice non è mai portabile. Noi usiamo **Intel x86**.
- Un processore a 64 bit è diverso da uno a 32 bit: quello a 32 bit è più comprensibile, mentre quello a 64 bit presenta molte più limitazioni (e complicazioni). Per questo ci concentriamo sul modello a 32 bit.
- Esiste un sottosistema che traduce i nostri input in sequenze di bit.
- La **memoria principale** contiene sia **dati** sia **programmi**.

La ALU

Dal punto di vista funzionale, la ALU (*Arithmetic Logic Unit*, unità aritmetico-logica) si occupa di:

- **operazioni logiche**;
- **operazioni aritmetiche** in base 2 o in complemento a 2.

(La FPU, unità per la virgola mobile, da studiare autonomamente se c'è tempo.)

Struttura di un calcolatore

Spazio di memoria

Lo spazio di memoria è formato da 2^{32} locazioni contigue, da un byte ciascuna, per un totale di 2^{32} byte $= 4$ GiB indirizzabili.

- Per accedere a un singolo byte bisogna specificare il suo **indirizzo**.
- Un dato che occupa più byte (ad esempio 2 o 4) è identificato dall'**indirizzo più piccolo** tra quelli coinvolti.
- In x86 vale la convenzione *little endian*: il byte **meno** significativo si trova all'indirizzo più piccolo e il byte **più** significativo all'indirizzo più grande.

Integrazione

Esempio: il valore a 32 bit `0x12345678` memorizzato a partire dall'indirizzo A occupa

indirizzo	A	$A+1$	$A+2$	$A+3$
contenuto	`78`	`56`	`34`	`12`

L'appunto originale parlava di "bit più significativi": il concetto corretto riguarda i *byte* più significativi.

Spazio di I/O

Lo **spazio di I/O** è lo spazio che il programmatore vede e che consiste nelle **interfacce** dei vari dispositivi.

- È formato da 2^{16} locazioni, dette **porte**, per un totale di 64k porte.
- Vi si accede con le istruzioni dedicate `IN` e `OUT`.

Differenza tra memoria e porte

Le locazioni di memoria sono **tutte equivalenti**: contengono semplicemente dati che si leggono e si scrivono. Le porte **no**: ciascuna è legata a un dispositivo specifico e il suo significato dipende dall'interfaccia a cui appartiene (leggere o scrivere una porta può avere effetti sul dispositivo).

— Fine Lezione 1 —

Questa lezione, come le sue precedenti e successive, può contenere errori e/o imprecisioni. Per il bene di tutti, è buona norma far notare all'autore l'errore o l'imprecisione così da risolvere nel minor tempo possibile. I canali di comunicazione, per eventuali avvisi generali o di errori trovati negli appunti, sono in cima al file. Grazie per la collaborazione e Buono studio!

Lezione 2 | 23 settembre 2026

I registri, EIP e ciclo di esecuzione, Codice macchina e codifica mnemonica, Formato delle istruzioni mnemoniche, Un esempio, Modalità di indirizzamento, Porte di I/O, Riepilogo delle notazioni

I registri

Un **registro** è una locazione di memoria **interna al processore**. I registri si dividono in:

- **generali**: usati nelle elaborazioni;
- **di stato**: servono per far andare avanti il processore. Sono `EIP` e `EFLAGS` (nelle note `EF`), che contiene i *flag*.

I flag

I primi due flag (`OF`, `SF`) hanno significato sugli **interi** (con segno), l'ultimo (`CF`) sui **naturali** (senza segno).

Flag	Vale 1 se...
`OF` (overflow)	durante l'ultima istruzione si è avuto un traboccamento, ossia il risultato non è rappresentabile (*interi*).
`SF` (sign)	il bit più significativo del risultato dell'ultima istruzione è 1, cioè è rappresentato un numero negativo (*interi*).
`ZF` (zero)	il risultato è composto da soli 0.
`CF` (carry)	l'ultima operazione ha generato un riporto o un prestito (*solo naturali*).

Registri generali

Sono 8 registri a 32 bit: `EAX`, `EBX`, `ECX`, `EDX`, `ESI`, `EDI`, `EBP`, `ESP`.

Con il passaggio all'architettura a 32 bit, i **16 bit più bassi** dei registri generali sono riferibili autonomamente. Di alcuni è possibile riferire anche parti da **8 bit**.

Integrazione

32 bit	16 bit	8 bit (alto)	8 bit (basso)
EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL
ESI	SI	–	–
EDI	DI	–	–
EBP	BP	–	–
ESP	SP	–	–

Le parti da 8 bit esistono solo per i primi quattro registri.

Alcuni registri sono **riservati a particolari funzioni**:

- `EAX`: operandi e risultati (*accumulatore*);
- `ESI`, `EDI`, `EBX`, `EBP`: *registri puntatore* (`B` = base, `I` = indice);
- `ESP`: indirizzamento della *pila* (stack).

Integrazione

Altri usi che incontreremo: `ECX` come contatore (negli esempi si usa `CL`) e `DX` per indirizzare le porte di I/O.

Nota

Se scrivo su un registro, il suo contenuto precedente viene **sovrascritto**: se mi serve ancora, devo averlo salvato altrove prima di riutilizzare il registro.

EIP e ciclo di esecuzione

`EIP` (o *program counter*) contiene l'**indirizzo di memoria della prossima istruzione da eseguire**.

Al **reset iniziale** la prima istruzione da eseguire si trova nella ROM (il *bootstrap*). Nelle note: `EIP` vale `0x0000FFF0`.

Integrazione

Nei processori x86 reali, al reset il registro di segmento `CS` ha base `0xFFFF0000`, quindi l'indirizzo fisico della prima istruzione è `0xFFFFFFF0`, cioè in cima ai 4 GiB, dove è mappata la ROM. Il valore `0x0000FFF0` è quello contenuto in `EIP`.

Perciò il processore:

1. **preleva** dalla memoria, all'indirizzo contenuto in `EIP`, l'istruzione;
2. **incrementa** `EIP` (in modo che punti all'istruzione successiva);
3. **esegue** l'istruzione e ripete il procedimento per le successive.

Il procedimento è **sequenziale**, ma può essere modificato dal comando `JMP` (e dai salti condizionati), che scrive un nuovo valore in `EIP`.

Codice macchina e codifica mnemonica

Codice macchina $\neq$ codifica mnemonica.

- *Macchina*: serie di 0 e 1.
- *Mnemonica*: codifica semplice (ASCII), pensata per l'essere umano.

Non ci interessa scrivere in codifica macchina, poiché **non stiamo costruendo un assemblatore**. Inoltre **non scriveremo gli indirizzi di memoria, ma nomi simbolici**. Un indirizzo di memoria (4 byte) si può scrivere nel campo *displacement*.

Formato delle istruzioni mnemoniche

Un'istruzione ha 3 campi:

1. **Codice operativo** (*opcode*): l'operazione da eseguire.
2. **Suffisso di lunghezza**: la lunghezza degli operandi.
3. **Operandi**: possono essere
   - registri,
   - memoria (indirizzi),
   - I/O,
   - costanti.

   Se ne possono avere 0, 1 o 2 per istruzione.

Se gli operandi sono due, il primo è la **sorgente** (*source*) e **non viene mai modificato**; il secondo è il **destinatario**, che **viene sostituito con il risultato** dell'istruzione.

Nota

- Gli operandi devono essere **sempre della stessa lunghezza**.
- Il suffisso di lunghezza è **attaccato al codice operativo**; dopo uno spazio vengono gli operandi.
- Non è sempre necessario scrivere il suffisso (ad esempio quando almeno un operando è un registro, perché la lunghezza è dedotta dal registro).
- **I conti si fanno dentro il processore**: i dati in memoria vengono portati nel processore, elaborati e poi riscritti.

Integrazione

Nelle istruzioni aritmetiche il destinatario è anche un *ingresso*: `ADDL %EBX, %EAX` calcola EAX $\leftarrow$ EAX + EBX. Il suo vecchio valore partecipa al calcolo, poi viene sostituito dal risultato.

Sintassi generale

`OPCODE`*size* `source, destination`

Suffissi di lunghezza

Suffisso	Nome	Dimensione
`B`	byte	8 bit (1 byte)
`W`	word	16 bit (2 byte)
`L`	long	32 bit (4 byte)

Tipi di istruzioni

- **Operative**: modificano lo stato del processore; sono tante.
- **Di controllo**: cambiano il flusso del programma (salti e salti condizionati).

Essendo il linguaggio semplice e limitato, il lavoro del programmatore risulta più difficile.

Nota

Si programma in Assembler **con il manuale davanti**. L'assemblatore **non ci aiuta**: bisogna fare molta attenzione ai suffissi.

Un esempio

Dalle note: `CL` è il contatore e all'inizio vale 0. Il valore viene *shiftato* e ciò che esce finisce nel flag `CF`: se è 0 non succede nulla, se è 1 si incrementa il contatore. Il risultato finisce in `CL`.

- `L` sta per *Long*, cioè 32 bit.
- Sorgente scritta come **numero esadecimale senza $ → indirizzo di memoria**.
- Con il **$** davanti ci si riferisce a una **costante**.
- `CMP` = *compare*: confronta due operandi aggiornando i flag, senza modificarli.
- `JE` = *jump if equal*: salta se `ZF` = 1; altrimenti il salto condizionato non ha effetto e si prosegue in sequenza.
- `ADC` = *add with carry*: somma sorgente, destinatario e `CF`.

Integrazione: esempio ricostruito

Le note originali sono frammentarie. Questo programma (sintassi AT&T) conta i bit a 1 di una parola in memoria, in accordo con quanto descritto sopra; l'indirizzo `0x1000` è inventato.

```
        MOVB  $0, %CL          # contatore = 0
        MOVL  0x1000, %EAX     # senza $: indirizzo di memoria
ciclo:  CMPL  $0, %EAX         # ZF=1 se EAX = 0
        JE    fine             # salta solo se ZF=1
        SHRL  $1, %EAX         # il bit espulso va in CF
        ADCB  $0, %CL          # CL = CL + 0 + CF
        JMP   ciclo
fine:
```

Modalità di indirizzamento

`source` e `destination` sono individuati tramite le **modalità di indirizzamento**.

Indirizzamento di registro

L'operando è in un registro: 8 registri generali a 32 bit, oppure a 16 bit, oppure a 8 bit. I registri si scrivono con il prefisso `%`, ad esempio `%EAX`.

Indirizzamento immediato

Vale **solo per l'operando sorgente**: l'operando è una costante scritta nell'istruzione.

Nota

È assolutamente necessario inserire il `$` prima della costante, ad esempio `$5`. Senza il `$` il numero sarebbe interpretato come indirizzo di memoria.

Indirizzamento di memoria

L'operando (**o sorgente o destinatario**) è in memoria. L'indirizzo effettivo è

$$\text{indirizzo} = \big(\text{base} + \text{indice} \times \text{scala} \pm \text{displacement}\big) \bmod 2^{32}$$

- la **base** è un registro generale **a 32 bit**;
- l'**indice** è un registro generale a 32 bit;
- la **scala** vale 1, 2, 4 oppure 8 (dimensione in byte degli elementi);
- il **displacement** è una costante intera.

Quindi la sintassi completa è

`OPCODE`*`sfx`* $\pm$`disp(base,indice,scala)`

Alcune componenti possono mancare, e si ottengono i casi particolari:

- **Diretto**: si usa solo il *displacement* (l'indirizzo). Se la memoria sia sorgente o destinatario dipende dall'opcode.
- **Indiretto**: **non si scrive il displacement**, si mette tra parentesi il registro base, ad esempio `(%EBX)`. L'operando si trova nelle 4 locazioni consecutive a partire dall'indirizzo contenuto in `EBX` (*operando puntato da* `EBX`).
  Come faccio a sapere che l'operando occupa 4 byte? Dal **suffisso**: è assolutamente necessario specificarlo (`L`), perché il puntatore da solo non dice la dimensione dell'operando.
- **Base e displacement**: `disp(%EBX)`, l'indirizzo è il contenuto di `EBX` spostato di `disp` byte.
- **Con indice e scala**: lo spostamento è `indice`$\times$`scala` byte, utile per scorrere vettori.
- **Base, indice e displacement** insieme (il caso completo, detto anche *bimodificato* con displacement).

Se il risultato supera 2^{32} si **tronca**, scartando il bit più significativo (calcolo modulo 2^{32}).

Nota: due operandi in memoria

Non si possono scrivere due operandi che provengano entrambi dalla memoria. Almeno uno dei due deve essere un registro (o la sorgente una costante).

Porte di I/O

Le porte non si trattano come la memoria: non si possono fare operazioni (aritmetiche, `MOV`, ...) direttamente sulle porte, ma solo trasferimenti con le istruzioni dedicate `IN` e `OUT`. Ci sono due indirizzamenti:

- **Diretto**: il numero della porta è una costante a 8 bit (offset fino a 255).
- **Indiretto con registro puntatore**: si usa **solo il registro** `DX`, che contiene il numero della porta.

Integrazione

Nelle note l'indirizzamento indiretto era indicato con offset < 255, ma `DX` è a 16 bit e permette di raggiungere tutte le 2^{16} porte (0–65535): il limite a 255 vale solo per l'indirizzamento diretto. Esempi (il dato passa per `AL`/`AX`/`EAX`):

```
INB   $0x60, %AL       # diretto: porta 0x60
OUTB  %AL, (%DX)       # indiretto: porta il cui numero e' in DX
```

Riepilogo delle notazioni

Simbolo	Significato
`$`	costante (indirizzamento immediato)
`%`	registro
numero senza `$`	indirizzo di memoria
`B`, `W`, `L`	byte (8 bit), word (16 bit), long (32 bit)

— Fine Lezione 2 —

Questa lezione, come le sue precedenti e successive, può contenere errori e/o imprecisioni. Per il bene di tutti, è buona norma far notare all'autore l'errore o l'imprecisione così da risolvere nel minor tempo possibile. I canali di comunicazione, per eventuali avvisi generali o di errori trovati negli appunti, sono in cima al file. Grazie per la collaborazione e Buono studio!

Lezione 3 | 24 settembre 2026

Istruzioni operative di trasferimento, La pila (stack), Istruzioni aritmetiche, Moltiplicazione, Divisione, Estensione di campo

Istruzioni operative di trasferimento

Le istruzioni di **trasferimento** spostano dati:

- dallo spazio di memoria a un registro;
- da registro a registro;
- da I/O a un registro.

Non è possibile il trasferimento **da memoria a memoria** (si potrebbero usare le *istruzioni stringa*).

Integrazione

Valgono anche i trasferimenti nei versi opposti (da registro a memoria, da registro a I/O) e da costante a registro o memoria. L'unico vincolo è che memoria–memoria è esclusa.

Nota

Le istruzioni di trasferimento **non modificano i flag**.

Le principali istruzioni sono:

- `MOV`: copia un dato dalla sorgente al destinatario (ad esempio va in memoria e sposta il dato in un registro). La sorgente non viene modificata.
- `LEA` (*load effective address*): **fa un conto**, calcola l'indirizzo effettivo dell'operando di memoria e lo inserisce in un registro, **senza accedere alla memoria**.
- `XCHG` (*exchange*): **scambia** sorgente e destinatario.

Integrazione

Con `MOV` il destinatario viene sovrascritto (i registri non sono "trasparenti": scrivere distrugge il valore precedente), quindi per scambiare due valori servirebbe un registro temporaneo. `XCHG` lo fa in una sola istruzione. Differenza tra `MOV` e `LEA`:

```
MOVL  4(%EBX,%ECX,2), %EAX   # EAX <- contenuto della memoria
LEAL  4(%EBX,%ECX,2), %EAX   # EAX <- EBX + ECX*2 + 4 (l'indirizzo)
```

La pila (stack)

Esiste un registro del processore che punta al **top della pila**: `ESP`. Potremmo usarlo come registro generale, ma molto probabilmente otterremmo un *segmentation fault*.

A cosa serve

- **Chiamata di sottoprogramma**: l'istruzione che chiama un sottoprogramma modifica `EIP`, ma prima deve fare la `PUSH` dell'indirizzo da cui ripartire al termine del sottoprogramma: `EIP` viene salvato dentro la pila.
- **"Parcheggio"** di valori: i registri generali sono **pochi e indispensabili per determinati conti**, quindi se ne salva il contenuto nella pila e lo si ripristina dopo.

Integrazione

Le istruzioni che fanno la `PUSH` di `EIP` e il ripristino sono `CALL` e `RET`.

Funzionamento

La pila è una **zona di memoria gestita dagli indirizzi più alti verso quelli più bassi**: quando si inserisce un dato, `ESP` viene **decrementato**.

- `PUSH`: decrementa `ESP` (di 2 o 4 byte, in base alla dimensione dell'operando) e scrive l'operando all'indirizzo puntato da `ESP`.
- `POP`: copia il dato puntato da `ESP` nel destinatario e **poi** incrementa `ESP`.

```
PUSHL %EAX      # ESP <- ESP-4 ; [ESP] <- EAX
...             # uso EAX per altri conti
POPL  %EAX      # EAX <- [ESP]  ; ESP <- ESP+4
```

Inizializzazione

Dobbiamo **dichiarare una zona di memoria da usare come pila**, e `ESP` deve essere **inizializzato prima** di eseguire il nostro programma. Il valore iniziale di `ESP` è quello **subito oltre** l'ultimo indirizzo della zona (il più alto), perché la `PUSH` decrementa prima di scrivere.

PUSH e POP nel dettaglio

- La `PUSH` ha **un solo operando**, che è la **sorgente**, e deve essere a **16 o 32 bit**.
- La `PUSH` di un operando in memoria è di fatto un trasferimento memoria–memoria (verso la pila), ma è ammessa.
- La `POP` **toglie** un dato dalla pila e lo mette in un registro generale o in memoria.

Attenzione

Ogni `PUSH` deve avere la sua `POP` corrispondente: altrimenti la pila si "incasina" e si ha *segmentation fault*.

Istruzioni aritmetiche

Molte istruzioni aritmetiche lavorano **senza distinguere tra naturali e interi**: l'operazione sui bit è la stessa. Modificano i **flag** con **due algoritmi diversi**, a seconda del tipo di operando; quindi devo guardare flag diversi:

- **naturali** → `CF`;
- **interi** → `OF`.

Consiglio del professore

Rivedere l'algoritmo di aggiornamento dei flag.

Addizione (ADD)

Il riporto entrante è sempre su un bit.

- `CF` contiene l'**ultimo riporto** dell'operazione: se vale 1, il risultato non è rappresentabile con il numero di cifre disponibili. Va bene per i **naturali**; per gli interi non significa nulla.
- Per gli **interi** si usa l'*overflow flag* `OF`: la somma è fuori dall'intervallo di rappresentabilità.

Algoritmo per gli interi:

- **segni discordi** → `OF` vale sempre 0 (il risultato è sempre rappresentabile);
- **segni concordi** → il risultato è corretto se ha lo stesso segno degli operandi, e allora `OF` $= 0$; altrimenti `OF` $= 1$.

Incremento e decremento

`INC` *dest*: incrementa di 1 il destinatario. Esiste per motivi storici e strutturali. `INC dest` $\neq$ `ADD $1, dest`: la prima **occupa meno byte** (è più compatta).

`DEC` *dest*: l'operazione contraria (sottrae 1).

Integrazione

Un'altra differenza: `INC` e `DEC` **non modificano** `CF`, mentre `ADD` e `SUB` sì.

Sottrazione (SUB)

$$\text{destinazione} \leftarrow \text{destinazione} - \text{sorgente}$$

- Il **prestito uscente** viene inserito in `CF`. Per i naturali, se `CF` $= 1$ il risultato non è rappresentabile (non posso scrivere il risultato).
- **Interi**: se i due numeri hanno **segni concordi** ci si muove verso il centro dell'intervallo, quindi il risultato è sempre interno all'intervallo (rappresentabile). Se i segni sono **discordi**, il risultato è corretto solo se ha il **segno del minuendo**; altrimenti `OF` $= 1$.

Addizione e sottrazione con riporto

- `ADC` (*add with carry*): somma **sorgente, destinatario e** `CF`.
- `SBB` (*subtract with borrow*): fa la stessa cosa con la sottrazione.

Servono per la **somma di operandi a tanti bit**: si spezza la somma in blocchi da 32 bit, propagando il riporto da un blocco al successivo.

Integrazione: esempio

Somma a 64 bit, `EDX:EAX` $\leftarrow$ `EDX:EAX` + `ECX:EBX`:

```
ADDL  %EBX, %EAX     # parte bassa, genera il riporto in CF
ADCL  %ECX, %EDX     # parte alta + riporto
```

Negazione (NEG)

`NEG` calcola l'**opposto** di un operando (**se questo esiste**) e aggiorna `CF` per i naturali e `OF` per gli interi.

Integrazione

Per gli interi l'opposto non esiste per il valore minimo dell'intervallo (-2^{n-1}, che non ha il corrispondente positivo): in quel caso `OF` $= 1$. Per i naturali l'opposto esiste solo per lo 0.

Confronto (CMP)

`CMP` calcola destinazione − sorgente e **scarta il risultato**: serve solo ad aggiornare i flag. Dai flag si può capire se un numero è minore dell'altro.

Integrazione

Dopo `CMP` (destinazione − sorgente):

- uguali: `ZF` $= 1$;
- **naturali**: destinazione $<$ sorgente se e solo se `CF` $= 1$;
- **interi**: destinazione $<$ sorgente se e solo se `SF` $\neq$ `OF`.

Questi flag sono quelli letti dai salti condizionati.

Moltiplicazione

La moltiplicazione è **diversa tra naturali e interi**:

- `MUL` (*multiply*): **naturali**;
- `IMUL` (*integer multiply*): **interi**.

Un operando è **implicito** (nell'accumulatore): viene usato sia come moltiplicando sia come contenitore del risultato. Cambia a seconda della dimensione scelta (8, 16 o 32 bit). Il risultato ha **il doppio dei bit** degli operandi, quindi non può traboccare.

Dimensione	Operando implicito	Risultato
8 bit (`B`)	`AL`	`AX`
16 bit (`W`)	`AX`	`DX:AX`
32 bit (`L`)	`EAX`	`EDX:EAX`

Divisione

La divisione è molto difficile. Anche qui ci sono due istruzioni: `DIV` per i naturali e `IDIV` (*integer divide*) per gli interi, ciascuna in tre versioni (8, 16 e 32 bit). Si specifica **solo il divisore**: l'istruzione ha un solo operando (`DIV source`), mentre il dividendo è implicito.

Dimensione	Quoziente	Resto
8 bit	$\texttt{AL} = \frac{\texttt{AX}}{\text{source}}$	`AH`
16 bit	$\texttt{AX} = \frac{\texttt{DX:AX}}{\text{source}}$	`DX`
32 bit	$\texttt{EAX} = \frac{\texttt{EDX:EAX}}{\text{source}}$	`EDX`

(Il quoziente è la divisione intera; il resto è in un registro diverso.)

Il problema della dimensione del quoziente

L'architettura Intel vuole mettere il **quoziente in metà dei bit** del dividendo. Se il quoziente non ci sta, la divisione **non è fattibile**: il programma non può avanzare (errore di divisione) e non si può fare nulla. Lo stesso accade per la divisione per zero. Dobbiamo **ovviare noi**: usare la divisione **di dimensione maggiore** rispetto a quella che consideriamo corretta.

Azzerare sempre DX ed EDX

Nella divisione tra naturali bisogna **azzerare sempre** `DX` (o `EDX`) prima della divisione: il loro contenuto verrà comunque modificato successivamente (conterranno il resto).

Integrazione: esempio e `IDIV`

Divisione di due naturali a 32 bit (`a` e `b` sono nomi simbolici di locazioni di memoria):

```
        MOVL  $0, %EDX      # parte alta del dividendo = 0
        MOVL  a, %EAX       # dividendo
        DIVL  b             # EAX = quoziente, EDX = resto
```

Per `IDIV` la parte alta del dividendo non va azzerata: deve contenere l'**estensione di segno** del dividendo (vedi istruzioni di conversione più avanti), altrimenti un dividendo negativo verrebbe letto come un grande numero positivo.

Segno e approssimazione

Con `IDIV` valgono le stesse limitazioni.

- Il quoziente è approssimato all'intero più vicino allo zero (**troncamento**).
- Il **resto** ha il **segno del dividendo** e il suo modulo è minore di quello del divisore.

Integrazione

Esempio: $-7 \div 2$ dà quoziente -3 e resto -1, infatti $-7 = (-3) \cdot 2 + (-1)$.

Estensione di campo

Estendere il campo significa rappresentare un numero su più cifre (più bit).

- **Naturali**: si aggiungono **zeri all'inizio**.
- **Interi**: non si può fare lo stesso; si **ripete il bit più significativo** (estensione di segno). In alternativa, per un numero negativo x si può rappresentare il modulo $\overline{x}+1$ (dove $\overline{x}$ è il complemento bit a bit) con zeri in testa e poi cambiarne il segno, poiché $x = -(\overline{x}+1)$.

Se estendiamo numeri con meno bit possiamo **sommarli** a numeri più lunghi: si usano gli **operatori di conversione**.

Integrazione

Esempi di istruzioni di conversione (mnemonico Intel, tra parentesi l'equivalente AT&T):

- `CBW` (`CBTW`): `AL` → `AX` con segno;
- `CWD` (`CWTD`): `AX` → `DX:AX` con segno;
- `CWDE` (`CWTL`): `AX` → `EAX` con segno;
- `CDQ` (`CLTD`): `EAX` → `EDX:EAX` con segno (utile prima di `IDIVL`);
- `MOVZ`xy: copia estendendo con zeri (naturali); `MOVS`xy: copia estendendo il segno (interi), ad esempio `MOVZBL`, `MOVSWL`.

— Fine Lezione 3 —

Questa lezione, come le sue precedenti e successive, può contenere errori e/o imprecisioni. Per il bene di tutti, è buona norma far notare all'autore l'errore o l'imprecisione così da risolvere nel minor tempo possibile. I canali di comunicazione, per eventuali avvisi generali o di errori trovati negli appunti, sono in cima al file. Grazie per la collaborazione e Buono studio!

Lezione 4 | 25 settembre 2026

Traslazioni (shift) e rotazioni, Istruzioni logiche, Istruzioni di controllo, Istruzioni per l'esecuzione di sottoprogrammi, Altre istruzioni, Modalità utente e modalità sistema, Varie

Traslazioni (shift) e rotazioni

Le **traslazioni** spostano i bit di un operando di un certo numero di posizioni; il bit che esce dall'operando finisce nel flag `CF`. Il verso dello spostamento può essere a sinistra o a destra.

Traslazione a sinistra

- `SHL` (*shift logical left*): se non si specifica nulla trasla di **1** posizione, altrimenti trasla di un numero di posizioni pari a `source`. Entrano zeri da destra.
- Permette la **moltiplicazione per** 2^k: in pratica usare una `MUL` è sconveniente.
- Non serve guardare `CF` a ogni passo, perché viene **sovrascritto** a ogni spostamento di una posizione (cioè n volte): alla fine contiene solo l'ultimo bit uscito.
- La stessa operazione si trova con il codice operativo `SAL` (*shift arithmetic left*). Si usa *logical* per i naturali e *arithmetic* per gli interi, ma cambia poco in entrambi i casi.

Integrazione

`SHL` e `SAL` sono in realtà la *stessa* istruzione (stessa codifica macchina): la distinzione è solo di leggibilità. Il numero di posizioni può essere una costante o il registro `CL`:

```
SHLL  %EAX          # EAX <- EAX * 2
SHLL  $3, %EAX      # EAX <- EAX * 8
SHLL  %CL, %EAX     # EAX <- EAX * 2^CL
```

Il risultato è corretto solo se non c'è traboccamento: per i naturali basta che `CF` non abbia perso bit significativi, per gli interi il segno non deve cambiare.

Traslazione a destra

Conviene per le **divisioni per** 2^k, ma il quoziente ha un'**approssimazione diversa** rispetto a `DIV`/`IDIV`: l'approssimazione è sempre **verso sinistra** sulla retta dei numeri, cioè per difetto (verso $-\infty$).

- `SHR` (*shift logical right*): vale quanto detto per la traslazione a sinistra, applicato a destra. Divide per 2^{source}; entrano **zeri** da sinistra $\Rightarrow$ si usa **solo per i numeri naturali**.
- `SAR` (*shift arithmetic right*): divide l'**intero** ottenendo l'approssimazione per difetto del quoziente. Per mantenere il segno, da sinistra entra una copia del bit di segno.

Shift o DIV?

`DIV`/`IDIV` e shift danno lo **stesso risultato** se (e solo se) il dividendo non è negativo **oppure** il resto è nullo.

Integrazione

La differenza si vede solo con dividendi negativi e resto non nullo: $-7 \div 2$ con `IDIV` dà -3 (troncamento verso zero), mentre `SAR` di 1 posizione dà -4 (per difetto). Con $-8 \div 2$ i due metodi coincidono (-4).

Rotazioni

Le **rotazioni non buttano via bit**: il bit che esce da una parte **rientra dalla parte opposta** (e viene copiato anche in `CF`). Valgono allo stesso modo a destra e a sinistra (`ROL`, `ROR`).

Nella *rotate through carry* (`RCL`, `RCR`) è come se avessimo **un bit in più**: `CF` fa parte dell'anello di rotazione, quindi il bit che esce va in `CF` e il vecchio `CF` rientra dall'altra parte.

Istruzioni logiche

Le istruzioni logiche operano **bit a bit** e **modificano i flag**.

Integrazione

Eccezione: `NOT` **non** modifica i flag. `AND`, `OR` e `XOR` azzerano `CF` e `OF` e aggiornano `SF` e `ZF` in base al risultato.

Attenzione

Ricordiamoci che i singoli bit **valgono sempre qualcosa**: un'operazione logica su un operando può cambiarne il valore numerico.

NOT

Ha **solo l'operando destinatario** e calcola il **complemento** del destinatario (ne inverte i bit).

AND

Ha due operandi. Nel destinatario viene sostituito il risultato dell'**AND bit a bit** tra sorgente e destinatario. A cosa serve?

- **Testare i bit** di un operando: si usa una **maschera**, cioè un operando che ha tutti 0 tranne il bit di interesse (che vale 1). Dopo l'`AND`, `ZF` $= 1$ se il bit testato era 0.
- **Azzerare un bit** di un registro: anche qui si usa una maschera, con 0 in corrispondenza del bit di interesse e 1 negli altri.
- **Estensione di campo per operandi naturali**: ad esempio se volessi sommare `AL` ed `EBX`, operandi di lunghezza diversa (e quindi non sommabili): si fa l'`AND` di `EAX` con una maschera con gli 8 bit finali a 1, che azzera i 24 bit superiori.

Integrazione

```
        # test del bit 3 di EAX (su una copia, perche' AND modifica la
    destinazione)
        MOVL  %EAX, %EBX
        ANDL  $0x00000008, %EBX    # maschera: solo il bit 3 a 1
        JZ    bit_zero             # ZF=1 se il bit 3 era 0

        # azzeramento del bit 3 di EAX
        ANDL  $0xFFFFFFF7, %EAX

        # estensione di AL su 32 bit (naturali), poi somma con EBX
        ANDL  $0x000000FF, %EAX
        ADDL  %EBX, %EAX
```

Per testare un bit senza modificare nessun operando esiste l'istruzione `TEST`, che calcola l'`AND` e aggiorna solo i flag.

OR

Ha due operandi. Nel destinatario viene sostituito il risultato dell'**OR bit a bit** tra sorgente e destinatario.

Integrazione

Si usa tipicamente per **forzare a 1** alcuni bit con una maschera che ha 1 in corrispondenza dei bit di interesse, ad esempio `ORL $8, %EAX` pone a 1 il bit 3.

XOR

Ha due operandi e la stessa meccanica di `AND` e `OR`, ma con l'operazione logica XOR: il risultato vale **1 se i bit sono diversi**, **0 se sono uguali**. Per cosa lo usiamo?

- **Cambiare il valore di un bit** dentro un operando, ad esempio il bit più significativo (si usa una maschera con 1 sui bit da invertire):
  - $A \oplus 0 = A$
  - $A \oplus 1 = \overline{A}$
- **Azzerare i registri**: `XORL %EAX, %EAX` confronta il registro con se stesso, ottenendo 0 ovunque. È meglio di `MOV` perché non bisogna specificare alcuna costante e l'istruzione occupa poco spazio.

Integrazione

La codifica di `XORL %EAX, %EAX` occupa **2 byte**, contro i 5 byte di `MOVL $0, %EAX` (nelle note: "1 bit anziché 5 byte"). Attenzione che `XOR` modifica i flag (in questo caso `ZF` $= 1$), mentre `MOV` no.

All'esame

All'esame **non passano programmi corretti ma scritti male**: si evitano effetti collaterali, programmazione "a spaghetti" e simili.

Istruzioni di controllo

JMP

`JMP` ha **tantissimi formati**: solitamente si usa il primo tipo (vedere le dispense su GitHub). Consente di saltare ad un'altra parte del programma, ad esempio per realizzare cicli.

Salti condizionati (Jcc)

`J`*cond* (*jump if condition met*): si **testa una condizione**; se è vera si comporta come una `JMP`, altrimenti l'istruzione viene **ignorata completamente** e si prosegue in sequenza. Con questi salti si realizza il **branching** (implementazione di `if` ed `else`).

Integrazione: esempio di if/else

```
        CMPL  b, %EAX          # confronto EAX con b (interi)
        JLE   altrimenti       # se EAX <= b salta al ramo else
        ...                    # ramo "then"
        JMP   fine
altrimenti:
        ...                    # ramo "else"
fine:
```

Condizioni sui singoli flag

Si usano dopo operazioni che variano i flag.

Istruzione	Salta se...
`JZ`	`ZF` $= 1$
`JNZ`	`ZF` $= 0$
`JC`	`CF` $= 1$
`JNC`	`CF` $= 0$
`JS`, `JNS`	`SF` $= 1$, `SF` $= 0$
`JO`, `JNO`	`OF` $= 1$, `OF` $= 0$

Nota

Usare `JS` è inutile se si lavora con i naturali (il bit più significativo è una cifra, non un segno).

Confronto di naturali

Si usano **dopo una** `CMP` (e da nessun'altra parte, perché le altre istruzioni modificano i flag), che confronta **sorgente** e **destinatario**. Le condizioni si riferiscono al **destinatario** della `CMP` precedente.

Istruzione	Salta se...	Flag
`JE`	uguali	`ZF` $= 1$
`JNE`	diversi	`ZF` $= 0$
`JA`	*above*: destinatario **strettamente maggiore** del sorgente	`CF` $= 0$ e `ZF` $= 0$
`JAE`	maggiore o uguale	`CF` $= 0$
`JB`	*below*: destinatario **strettamente minore** del sorgente	`CF` $= 1$
`JBE`	minore o uguale	`CF` $= 1$ o `ZF` $= 1$

Confronto di interi

Anche questi si trovano **solo dopo una** `CMP`.

Istruzione	Salta se...	Flag
`JE`	come per i naturali	`ZF` $= 1$
`JNE`	come per i naturali	`ZF` $= 0$
`JG`	*greater*: destinatario **strettamente maggiore** del sorgente	`ZF` $= 0$ e `SF` $=$ `OF`
`JGE`	maggiore o uguale	`SF` $=$ `OF`
`JL`	*less*: destinatario **strettamente minore** del sorgente	`SF` $\neq$ `OF`
`JLE`	minore o uguale	`ZF` $= 1$ o `SF` $\neq$ `OF`

Integrazione: perché `SF` e `OF`

Per `JGE` (destinatario $\geq$ sorgente) ci sono due situazioni in cui `SF` $=$ `OF`:

- `OF` $= 0$ e `SF` $= 0$: la differenza è rappresentabile e non negativa;
- `OF` $= 1$ e `SF` $= 1$: la differenza **non è rappresentabile**, quindi il bit di segno ottenuto è sbagliato (invertito) e il risultato vero è non negativo.

Per questo, per gli interi, non basta guardare `SF`: bisogna confrontarlo con `OF`.

Istruzioni per l'esecuzione di sottoprogrammi

Entrambe le istruzioni seguenti fanno riferimento alla **pila**.

CALL

`CALL` **salta ad un sottoprogramma**; ha gli stessi formati di `JMP`.

1. **Salva in pila** il valore corrente di `EIP`, cioè l'indirizzo dell'istruzione successiva alla `CALL`;
2. **sostituisce** il valore di `EIP` con l'indirizzo della prima istruzione del sottoprogramma.

RET

`RET` **ritorna al programma chiamante**. Si trova in fondo a un sottoprogramma ed è analoga al *return* dei linguaggi ad alto livello. Il suo formato è solo `RET`.

1. esegue una `POP` dalla pila di **4 byte**;

2. **sostituisce** con questo valore il contenuto di `EIP`.

Dopo la `RET` l'esecuzione prosegue con l'**istruzione di rientro**, cioè quella che segue la `CALL` nel programma chiamante.

Integrazione: esempio

```
        CALL  sottoprog        # push dell'indirizzo di rientro, EIP <-
    sottoprog
        ...                    # istruzione di rientro
        ...
sottoprog:
        ...                    # corpo del sottoprogramma
        RET                    # pop in EIP: si torna alla riga dopo la CALL
```

Dentro il sottoprogramma ogni `PUSH` deve avere la sua `POP`: se `ESP` non punta all'indirizzo di rientro quando si esegue `RET`, si carica in `EIP` un valore sbagliato.

Altre istruzioni

- `NOP`: non fa proprio nulla.
- `HLT` (*halt*): fa **terminare l'attività del processore**. Non è possibile usarla in modalità utente.

Modalità utente e modalità sistema

Il processore Intel ha **due modalità** di funzionamento:

- **utente**;
- **sistema**: contiene anche le istruzioni **privilegiate**.

Istruzioni privilegiate: `HLT`, `IN`, `OUT`.

Se si usano `IN` e `OUT` in modalità utente viene sollevata una **eccezione**, che può ad esempio **inchiodare** il programma.

Per usarle è necessario passare per **sottoprogrammi di sistema** (routine del sistema operativo): se potessimo usarle direttamente, potremmo indurre uno **stato di inconsistenza** nella macchina.

Meccanismo di interruzione

Il **meccanismo di interruzione** avviene quando si passa a un sottoprogramma **non definito da noi**: il processore salta a una routine di sistema e, al termine, il controllo torna al programma interrotto.

Integrazione

Le cause tipiche sono: le *eccezioni* (come l'uso di un'istruzione privilegiata o una divisione non fattibile), le richieste dei dispositivi di I/O e le chiamate esplicite al sistema operativo (istruzione `INT`).

Varie

- **GAS** (*GNU Assembler*): documentazione su `www.delorie.com`.
- Venerdì prossimo: lezione con l'assistente di laboratorio.

— Fine Lezione 4 —

Questa lezione, come le sue precedenti e successive, può contenere errori e/o imprecisioni. Per il bene di tutti, è buona norma far notare all'autore l'errore o l'imprecisione così da risolvere nel minor tempo possibile. I canali di comunicazione, per eventuali avvisi generali o di errori trovati negli appunti, sono in cima al file. Grazie per la collaborazione e Buono studio!

Lezione 5 | 29 settembre 2026

Struttura di un programma assembler, Direttive, Costanti numeriche, Controlli di flusso, La programmazione a spaghetti, Stringhe, Gestione dei sottoprogrammi (ripresa), Gestione dell'I/O in assembler

Avviso

Da venerdì: esercitazioni del dr. Zippo.

Struttura di un programma assembler

Un programma assembler è diviso in **2 sezioni**:

- **Dati**: contiene gli indirizzi e i nomi simbolici delle variabili. Niente obbliga a metterla all'inizio del codice, **ma fare altrimenti sarebbe programmare male**.
- **Codice**: contiene le istruzioni del programma.

Integrazione

Nelle note la sezione codice era descritta come "primitive che servono per assemblaggio e dichiarazioni delle variabili". Le *primitive* (o *direttive*) sono comandi rivolti all'**assemblatore**, non al processore: non producono istruzioni eseguibili. Le dichiarazioni delle variabili stanno nella sezione dati.

Righe e direttive di struttura

Un'istruzione **occupa una riga** e termina con CR (a capo). Una riga contiene:

- un **nome** (etichetta), seguito da :;
- una **keyword** seguita dagli **operandi**.

Direttiva	Significato
`.global main`	dichiara il **sottoprogramma globale** `main`, che viene chiamato dall'ambiente
`.data`	dà inizio alla **sezione dati** (l'insieme dei dati)
`.text`	dà inizio alla porzione di **codice**

- Per motivi legati al **debugger** (breakpoint ed esecuzione) conviene che la **prima istruzione sia una** `NOP`.

- L'assemblatore è **case-insensitive** per quanto riguarda le proprie **direttive**, ed è invece **case-sensitive** per i **nomi**.
- **Bit più significativi → indirizzo più alto** (convenzione *little endian*).
- Posso dare **più alias allo stesso byte**, ma è alquanto sconveniente.

Le due passate

L'assemblatore fa **due passate**: nella prima controlla il programma, nella seconda lo traduce. Per questo è possibile usare un salto verso un'istruzione **non ancora definita** (*riferimento in avanti*).

Integrazione

Più precisamente: nella prima passata l'assemblatore costruisce la *tabella dei simboli* (a ogni nome associa il suo indirizzo); nella seconda genera il codice macchina, usando la tabella per risolvere anche i nomi definiti più avanti.

Integrazione: schema di programma

```
        .global main
        .data
a:      .word   5               # variabile scalare da 2 byte
msg:    .asciz  "Ciao\n"
        .set    lung, . - msg   # costante simbolica: lunghezza di msg

        .text
main:   NOP                     # prima istruzione: nop (per il debugger)
        MOVW    a, %AX
        ...
        RET                     # ritorno all'ambiente
```

Il carattere `#` introduce un commento fino a fine riga.

Direttive

Una direttiva ha la forma `KEYWORD operandi` seguita da CR.

Dichiarazione di variabili

- **Variabile scalare**: `nome:` `.word` *valore* (una `.word` vale **2 byte**).
- **Variabile vettoriale**: `nomealtravariabile:` `.byte` *valore1*`,` *valore2* è un vettore con due componenti da 1 byte ciascuna.
- Direttive analoghe: `.byte` (1 byte), `.word` (2 byte), `.long` (4 byte).

Integrazione

Gli operandi di `.byte`, `.word`, `.long` sono i *valori iniziali* (espressioni) delle componenti, non indirizzi come scritto nelle note originali.

È possibile anche avere una variabile che si ottiene dalla **somma di un indirizzo e di un numero di celle di memoria** (da dispense del professore):

```
var4:   .long   var3 + 2        # var4 contiene l'indirizzo var3 + 2
```

.fill

`.fill` *numero*`,` *dim*`,` *espressione*: riserva *numero* elementi, ciascuno di *dim* byte e di valore *espressione*.

- *dim* = 1, 2 o 4 corrisponde a byte, word o long, a seconda del numero di byte.
- Se non viene specificata la dimensione si assume 1 byte; se manca il valore si assume 0.

Integrazione

Nelle note *dim* era detta "dimensione in bit" con default 0. In GAS la dimensione è in **byte**; i default sono 1 per la dimensione e 0 per il valore. Esempio: `buffer:  .fill 10, 1, 0` riserva 10 byte a zero.

Stringhe di caratteri

- `.ascii`: permette di inserire un messaggio tra virgolette `" "`.
- `.asciz`: come `.ascii`, ma **aggiunge un carattere nullo** (`0`) in fondo, come le stringhe del C.
- I **caratteri speciali** sono gli stessi del C++ (`\n`, `\t`, ...).

.include e .set

- `.include`: prende come operando un **percorso** (path) e include un file sorgente nel file che stiamo scrivendo (utile ad esempio per includere sottoprogrammi).
- `.set`: associa un **nome** a un'**espressione**, creando **costanti simboliche più leggibili**. Può essere usata anche per il **calcolo della memoria occupata**, come nell'esempio sopra con `lung`, dove `.` indica l'indirizzo corrente.

Nota

Le costanti simboliche è bene metterle **dentro la sezione dati**.

Costanti numeriche

Naturali

I naturali **non hanno segno** e vengono **convertiti in base 2** (nelle note "base 3", è un refuso: in memoria i numeri sono sequenze di bit). Possiamo scriverli in base 2, 8, 10 o 16:

Base	Prefisso	Esempio (valore 15)
2	`0b`	`0b1111`
8	`0`	`017`
10	nessuno	`15`
16	`0x`	`0xF`

In base 10 non deve esserci alcun prefisso: un numero che inizia con `0` è letto come ottale (a maggior ragione, lo zero stesso va bene in qualsiasi base).

Integrazione

I numeri **negativi** si scrivono con il segno meno (ad esempio `-5`) e sono memorizzati in complemento a 2.

Dimensione sbagliata

Che succede se la costante non ha la dimensione giusta?

- se è **più piccola**, viene **estesa** (ma l'assemblatore non lo segnala);
- se è **più grande**, viene **troncata** (e, se siamo fortunati, l'assemblatore ci avvisa).

Attenzione

Non contare troppo sull'assemblatore per la scrittura di programmi corretti.

Controlli di flusso

I controlli di flusso vengono implementati **attraverso le istruzioni** `JMP` (e i salti condizionati).

if / else

L'`if` e l'`else` si scrivono in maniera **invertita** rispetto all'ordine di esecuzione a cui siamo abituati in C++ (si usa ad esempio una `JB`): **prima il ramo else, poi il ramo then**.

```
        MOVL  a, %EAX
        CMPL  b, %EAX          # EAX - b
        JB    then             # se a < b (naturali) salta al ramo then
        ...                    # ramo else (si arriva qui se non si salta)
        JMP   fine
then:   ...                    # ramo then
fine:
```

Ciclo for

Un ciclo `for` sfrutta una `JMP` (per **reiterare** il ciclo) e una `JE` (per **terminarlo**).

```
        MOVL  $0, %EBX
ciclo:  CMPL  $10, %EBX
        JE    fine             # esce quando EBX = 10
        ...                    # corpo del ciclo
        INCL  %EBX
        JMP   ciclo
fine:
```

Ciclo do-while

Un `do-while` richiede una `CMP` e una `JB` (che riporta all'inizio del corpo finché la condizione vale).

```
ripeti: ...                     # corpo del ciclo
        CMPL  b, %EAX
        JB    ripeti            # ripete finche' EAX < b (naturali)
```

L'istruzione LOOP

`LOOP` *etichetta*: **decrementa** `ECX` e **salta se** `ECX` $\neq 0$. È il modo migliore per rappresentare un ciclo `for`.

```
        MOVL  $10, %ECX         # numero di iterazioni
ciclo:  ...                     # corpo (ECX non va toccato)
        LOOP  ciclo             # ECX <- ECX-1 ; se ECX != 0 salta
```

- Se qualcuno modifica `ECX` dentro il ciclo, si scrivono istruzioni **incomprensibili**.
- Per convenzione si usa `ECX` **solo se il ciclo è a scendere**. Se è a salire, **è necessario usare un altro registro**, ad esempio `EBX`.
- `LOOP` non modifica i flag.

Integrazione

Se `ECX` vale 0 all'ingresso, il primo decremento lo porta a $2^{32} - 1$ e il ciclo viene eseguito circa 4 miliardi di volte: il controllo che `ECX` sia diverso da 0 va fatto prima.

LOOPE e LOOPNE

Esistono anche le versioni **condizionali**: `LOOPE` e `LOOPNE`. Hanno senso solo se usate **dopo una** `CMP` (o un'altra istruzione che aggiorna `ZF`). Si usano per cicli con un **numero massimo di iterazioni** da cui si può **uscire prima**.

Integrazione

Decrementano `ECX` e saltano se `ECX` $\neq 0$ *e* `ZF` $= 1$ (`LOOPE`) oppure `ZF` $= 0$ (`LOOPNE`).

Le istruzioni `LOOP` **non sono indispensabili**: lo stesso ciclo si può scrivere con `JMP` e salti condizionati.

La programmazione a spaghetti

La programmazione a spaghetti è **da evitare**, perché **viola la verificabilità** dei programmi.

Integrazione

Succede ad esempio quando un ciclo interno interferisce con quello esterno (salti che entrano o escono a metà dei cicli, oppure stesso registro contatore): il ciclo più esterno perde il suo significato. Con `LOOP` annidate, il ciclo interno porta `ECX` a 0, quindi il contatore esterno va salvato con una `PUSH` prima del ciclo interno e ripristinato con una `POP` dopo, oppure si usa un altro registro.

Stringhe

Le **stringhe sono vettori**: in assembler consentono di scrivere **cicli** e **accessi sequenziali in memoria** (ad esempio un puntatore che avanza di elemento in elemento). Per questi accessi esistono anche le istruzioni stringa citate nella lezione sul trasferimento dati.

Gestione dei sottoprogrammi (ripresa)

In assembler non esiste l'idea di variabile locale.

È necessario **stabilire e rispettare convenzioni** per il passaggio dei parametri. Ci sono 3 modi:

1. usare **locazioni di memoria condivise** (non ottimale);
2. sfruttare i **registri**;
3. passare i parametri **attraverso la pila**: è **particolarmente sconsigliato** perché difficile da seguire e verificare: dimenticare anche una minima cosa sarebbe un problema.

Chi scrive un sottoprogramma **deve specificare nei commenti** i **parametri di ingresso e di uscita** (il professore se lo aspetta, perché è il modo giusto di programmare).

Integrazione: esempio di commento

```
# sottoprog: calcola il massimo di due naturali
# ingresso:  EAX = primo numero, EBX = secondo numero
# uscita:    EAX = massimo
# registri modificati: nessuno oltre a EAX
```

Effetti collaterali nei registri

Gli effetti collaterali sui registri **non devono assolutamente succedere**: all'esame ci si gioca dei punti.

Conviene che i sottoprogrammi abbiano una **prefazione** fatta di `PUSH` e di `POP`, che permettono ai calcoli di usare i registri (che altrimenti verrebbero "sporcati"). La pila va lasciata **in ordine**, altrimenti **segmentation fault**.

Da verificare

Nelle note la frase specifica che le `PUSH` e le `POP` "devono essere fatte dal chiamante", ma la parola "prefazione" fa pensare a un'operazione all'inizio del sottoprogramma (quindi del chiamato). Qualunque sia la scelta, la convenzione deve essere *una sola* e rispettata sempre, e ogni registro salvato deve essere ripristinato. Conviene controllare sulle slide quale delle due vale.

Integrazione

Il registro che contiene il risultato (di solito `EAX`, vedi sotto) è l'unica modifica ammessa e va dichiarata nei commenti.

Il simbolo ???

Scrivere `???` nel codice è un trucco che **impedisce l'assemblazione finché non metto mano al codice**: si lascia al posto di una parte ancora da scrivere, e l'assemblatore dà errore finché non la sostituisco.

Convenzioni

- Il registro `EAX`, al momento del ritorno, deve contenere un **codice** che dice se il risultato è corretto: **0 significa ok**, gli altri numeri rappresentano **codici di errore**. (Non è obbligatorio in vista dell'esame.)
- **Pila** (cose che vanno assolutamente fatte):
  1. si **dichiara una pila**, riservando memoria: lo deve fare **il programmatore**, ma nel nostro ambiente se ne occupa l'ambiente;
  2. si **inizializza** `%ESP` al valore **successivo al fondo della pila** (il primo indirizzo subito oltre la zona riservata); altrimenti contiene il valore lasciato dall'ultima persona che ci ha messo le mani.

Le cose aggressivamente ricorsive non vanno bene in assembler.

Gestione dell'I/O in assembler

Si sfruttano i **servizi del sistema DOS** attraverso una **libreria**. Tutto avviene "a suon di codifiche ASCII".

Carattere	Codice	Effetto
Ritorno carrello (CR)	`0x0D`	il cursore ritorna all'inizio della riga
Avanzamento linea (LF)	`0x0A`	il cursore avanza di una riga

- **In uscita** (stampa a video): escono le codifiche ASCII **dei singoli caratteri**, che vengono stampati uno dietro l'altro.
- **In ingresso** vale la stessa cosa: arrivano i **singoli caratteri**, che dobbiamo **concatenare** noi.

Integrazione

Per i numeri, la conversione è a carico del programmatore. Le cifre `'0'`–`'9'` hanno codici ASCII `0x30`–`0x39`: in ingresso si sottrae `0x30` a ogni carattere letto e si calcola valore $\leftarrow 10 \cdot$ valore $+$ cifra; in uscita si ricavano le cifre una alla volta (con divisioni per 10) e si somma `0x30` a ciascuna prima di stamparla.

— Fine Lezione 5 —

Questa lezione, come le sue precedenti e successive, può contenere errori e/o imprecisioni. Per il bene di tutti, è buona norma far notare all'autore l'errore o l'imprecisione così da risolvere nel minor tempo possibile. I canali di comunicazione, per eventuali avvisi generali o di errori trovati negli appunti, sono in cima al file. Grazie per la collaborazione e Buono studio!

Lezione 6 | 30 settembre 2026

La libreria `utility`: I/O di caratteri e stringhe, Input/output di numeri esadecimali, Input/output di numeri decimali, Manipolazione di stringhe, Efficienza della programmazione assembly

La libreria `utility`: I/O di caratteri e stringhe

Per permettere l'I/O di caratteri e stringhe è necessario includere il file **utility**:

```
        .INCLUDE "./files/utility.s"
```

Nota

Sono **sottoprogrammi**, perciò è necessario **chiamarli** (`CALL`).

Sottoprogramma	Funzione
`inchar`	legge un carattere **senza fare l'eco** sul video e inserisce la sua codifica ASCII in `%AL`.
`outchar`	**stampa a video** la codifica ASCII di un carattere.
`newline`	va a capo: stampa il **ritorno carrello** e il **line feed** (vedi lezione del 29 settembre).
`pausen`	**mette in pausa** il programma e stampa il *checkpoint* (per il debug): ci permette di capire dove si inchioda il nostro programma.
`inline`	permette l'inserimento di un'**intera riga** di caratteri da terminale (buffer di 80 caratteri) e fa l'**eco** sul video durante l'inserimento. Termina anche quando si preme invio.
`outline`	come `inline`, ma per l'**output**.
`outmess`	stampa un messaggio: `%EBX` deve contenere l'**indirizzo di memoria del buffer** e `%CX` il **numero di caratteri** da stampare a video.

Nota

- `outchar` usa internamente dei registri; per questo è scritto con delle `PUSH` e delle `POP` che li salvano e li ripristinano, al fine di "mantenere l'ordine": il chiamante non vede registri modificati.
- In realtà `inline` accetta **al massimo 78 caratteri** utili, perché nel buffer da 80 trovano posto anche il line feed e il ritorno carrello.

Integrazione: esempio

```
        CALL  inchar            # AL <- codice ASCII del carattere letto
        CALL  outchar           # stampa il carattere (passato in AL)
        CALL  newline           # va a capo

        LEAL  msg, %EBX         # EBX = indirizzo del messaggio
        MOVW  $lung, %CX        # CX  = numero di caratteri da stampare
        CALL  outmess
```

Il fatto che `outchar` riceva il carattere in `%AL` è dedotto per simmetria con `inchar`; i registri richiesti da `inline` e `outline` non sono indicati negli appunti: vanno verificati nelle dispense.

Input/output di numeri esadecimali

I sottoprogrammi hanno nomi diversi a seconda del **prefisso** e operano su numeri con una quantità diversa di byte:

Input	Caratteri	Registro
`inbyte`	2	`%AL`
`inword`	4	`%AX`
`inlong`	8	`%EAX`

Vale lo stesso per gli **output**, con prefisso `out` (`outbyte`, `outword`, `outlong`).

Attenzione

È assolutamente necessario **riempire di 0 le cifre iniziali** se la codifica non le occupa tutte: ad esempio, per scrivere 3 in `inbyte` bisogna scrivere `03`.

Input/output di numeri decimali

Anche qui i nomi dipendono dal **prefisso** e dalla dimensione:

Input	Caratteri	Registro
`indecimal_byte`	3	`%AL`
`indecimal_word`	5	`%AX`
`indecimal_long`	10	`%EAX`

Vale lo stesso per gli **output**, con prefisso `outdecimal`.

Attenzione

Anche qui bisogna **riempire di 0 le cifre iniziali** non usate: per scrivere 3 in `indecimal_byte` si scrive `003`.

Integrazione

Negli appunti i caratteri del formato `long` erano 8 e l'esempio era `03`: sembra una copia della sezione esadecimale. Il massimo naturale a 32 bit, $2^{32} - 1 = 4\,294\,967\,295$, ha **10 cifre decimali**, quindi 8 non basterebbero; allo stesso modo il byte ha 3 cifre (da `000` a `255`). Il numero esatto di caratteri è da verificare nelle dispense. Nelle note originali si parlava di "bit" da riempire: sono in realtà *cifre*.

Manipolazione di stringhe

Nota

L'assembler non ha **né tipi di dati** né **strutture dati**, ma supporta comunque il concetto di **vettore** (che prende il nome di *stringa*).

Il prefisso REP

`REP` è un **prefisso di ripetizione**: può essere usato **solo con le istruzioni stringa**. Funziona con **una singola istruzione**, non con un blocco di codice.

Integrazione

`REP` ripete l'istruzione stringa finché `ECX` $\neq 0$, decrementando `ECX` a ogni ripetizione: il numero di ripetizioni va quindi caricato in `ECX` prima.

Le istruzioni stringa

Le istruzioni stringa hanno un suffisso di lunghezza (`B`, `W`, `L`) e operano su un elemento da 1, 2 o 4 byte. I puntatori sono `%ESI` (sorgente) ed `%EDI` (destinazione).

Istruzione	Operazione	Puntatori aggiornati
`MOVS`*suf*	copia l'elemento puntato da `ESI` nella locazione puntata da `EDI`	`ESI`, `EDI`
`LODS`*suf*	carica in `AL`/`AX`/`EAX` l'elemento puntato da `ESI`	`ESI`
`STOS`*suf*	scrive `AL`/`AX`/`EAX` nella locazione puntata da `EDI`	`EDI`
`CMPS`*suf*	**confronta due stringhe**: confronta gli elementi puntati da `ESI` ed `EDI`, aggiornando i flag	`ESI`, `EDI`
`SCAS`*suf*	confronta il contenuto di un registro (`AL`/`AX`/`EAX`) con la locazione di memoria indirizzata da `EDI`: serve a cercare un **valore noto dentro un vettore**	`EDI`

- Ogni istruzione incrementa i puntatori **di 1, 2 o 4**, a seconda del suffisso (cioè del registro usato).
- **Non è necessario preoccuparsi dell'aggiornamento dei puntatori**: lo fa l'istruzione.
- Le istruzioni stringa permettono inoltre di "**spianare**" un intero buffer di memoria (porre tutto a 0).
- `LODS` **sovrascrive** il registro a ogni passo, quindi non va bene dopo `REP`: alla fine resterebbe solo l'ultimo elemento.

MOVL o MOVS?

Nelle note originali si scriveva "`MOVL` è un'istruzione stringa": l'istruzione stringa è `MOVS` con suffisso (`MOVSB`, `MOVSW`, `MOVSL`). `MOVL` è la normale `MOV` a 32 bit.

Integrazione: esempi

```
# copia di 10 long da vettA a vettB
CLD                     # DF = 0: avanti
LEAL  vettA, %ESI
LEAL  vettB, %EDI
MOVL  $10, %ECX
REP   MOVSL

# azzeramento di un buffer di 20 long (80 byte)
CLD
MOVL  $0, %EAX
LEAL  buffer, %EDI
MOVL  $20, %ECX
REP   STOSL
```

La direction flag

La **direction flag** `DF` è un flag presente nel **registro dei flag**. Si manipola con:

- `STD` (*set direction flag*): `DF` $= 1$, i puntatori vengono **decrementati**, cioè si copia **a ritroso**;
- `CLD` (*clear direction flag*): `DF` $= 0$, i puntatori vengono incrementati, cioè si procede **in avanti**.

Per cosa la usiamo:

- *compare* e *scan*;
- *copia*, nel caso di elementi **sovrapposti in memoria** (sorgente in *overlapping* con destinatario): se non si tiene conto dell'orientamento di `DF`, avverrebbero **sovrascritture** di dati ancora da copiare.

Integrazione

Per una copia con sovrapposizione: se la destinazione è a un indirizzo *maggiore* della sorgente si copia a ritroso (`STD`, partendo dall'ultimo elemento); se è minore, in avanti (`CLD`).

REPE e REPNE

`CMPS` e `SCAS` possono essere usate dopo un'istruzione di ripetizione, **ma non** `REP`: servono `REPE` e `REPNE`.

Cosa fa `REPE`?

1. se `ECX` $= 0$ termina;
2. altrimenti **decrementa** `ECX`;
3. esegue l'istruzione stringa **per intero**;
4. torna alla condizione di testa (punto 1).

Integrazione

Il ciclo ha anche una seconda condizione di uscita, che dipende da `ZF` (aggiornato dopo ogni passo di `CMPS`/`SCAS`):

- `REPE` (*repeat while equal*) continua finché `ZF` $= 1$ e termina appena `ZF` $= 0$;
- `REPNE` (*repeat while not equal*) continua finché `ZF` $= 0$ e termina appena `ZF` $= 1$.

Per questo, al termine, `ZF` dice *perché* il ciclo è finito. Esempio di ricerca di un valore:

```
        CLD
        MOVB  $0x41, %AL        # valore cercato ('A')
        LEAL  vett, %EDI
        MOVL  $100, %ECX        # massimo numero di elementi da guardare
        REPNE SCASB             # si ferma se AL = elemento (ZF=1) o ECX = 0
        JE    trovato           # ZF=1: valore trovato
```

Modalità sistema

Esistono le istruzioni `INS`*suf* e `OUTS`*suf*: `INS`*suf* fa un trasferimento **da I/O a memoria**, `OUTS`*suf* il contrario, da memoria a I/O.

Integrazione

Sono istruzioni di I/O, quindi **privilegiate** (vedi la lezione del 25 settembre): non si possono usare in modalità utente.

All'esame

A meno che l'uso delle istruzioni stringa non venga richiesto esplicitamente, bisogna **valutare sempre se siano utili** nel nostro codice. **Rispettare sempre le specifiche.**

Efficienza della programmazione assembly

L'assembler **non ottimizza il codice**: dobbiamo farlo noi.

Come si fa a inferire il tempo di esecuzione di un processo? Non è possibile. Dipende da:

- i **dati**;
- lo **stato del sistema**;
- **chi usa il processore**: altri utenti, o il sistema operativo stesso, possono occupare la CPU.

Un calcolatore non è una macchina fatta per eseguire in tempo prevedibile.

Correttezza	vincolo
Tempo di esecuzione	funzione obiettivo

Cioè: tra tutti i programmi *corretti* si cerca quello più veloce.

Cosa impedisce di calcolare il tempo di esecuzione

- Il **clock non va a velocità costante**.

- Alcuni **meccanismi variano il normale ordine di svolgimento**:
  - **cache** (memoria veloce tra processore e memoria principale);
  - **code di prefetch** (istruzioni lette in anticipo);
  - **esecuzione in pipeline**: parti del processore dedicate che lavorano a catena di montaggio;
  - l'esecuzione **non è sequenziale**;
  - il processore **scommette** (*branch prediction*): prevede l'esito dei salti condizionati.

Lunghezza delle istruzioni e tempo di fetch

Il tempo di fetch dipende dal **numero di byte** occupati dall'istruzione, cioè dall'**opcode** e dal **posizionamento degli operandi in memoria**. Ecco perché è meglio `XORL %EAX, %EAX` di `MOVL $0, %EAX` (2 byte contro 5, vedi la lezione del 25 settembre).

Costo delle istruzioni

Istruzioni	Costo (ordine di grandezza)
operative della ALU	1 clock
moltiplicazioni e divisioni	circa 10 clock
trascendenti	circa 100 clock
di controllo	molto costose

Integrazione

I salti sono costosi perché, se la previsione del processore è sbagliata, la pipeline va svuotata e riempita di nuovo.

Evitare moltiplicazioni e divisioni

Poiché moltiplicazioni e divisioni costano tanto, possono essere evitate:

- usando lo **shift** (moltiplicazioni e divisioni per 2^k, vedi la lezione del 25 settembre);
- sfruttando l'**uso improprio** (*hijacking*) della `LEA`: poiché `LEA` calcola un indirizzo senza accedere alla memoria, può servire per fare somme e moltiplicazioni per 2, 4, 8.

Integrazione

```
LEAL  (%EAX,%EAX,4), %EBX     # EBX <- EAX + 4*EAX = 5*EAX
LEAL  3(%EAX,%EAX,2), %EBX    # EBX <- 3*EAX + 3
```

Curiosità

Cercare su Google **movfuscator**: un compilatore che traduce programmi C usando *solo* istruzioni `MOV`.

In assembler scriviamo "pezzi di codice piccini e indispensabili".

— Fine Lezione 6 —

Questa lezione, come le sue precedenti e successive, può contenere errori e/o imprecisioni. Per il bene di tutti, è buona norma far notare all'autore l'errore o l'imprecisione così da risolvere nel minor tempo possibile. I canali di comunicazione, per eventuali avvisi generali o di errori trovati negli appunti, sono in cima al file. Grazie per la collaborazione e Buono studio!

Lezione 7 | 1 ottobre 2026

Differenze tra processori a 32 e a 64 bit, Modifiche generali all'assembler, MOVABS, Moltiplicazioni, divisioni e altre istruzioni, Sistema fisico e modello, Definizione di rete logica, I limiti del modello, Le reti combinatorie, Reti combinatorie per numero di ingressi

Parte I

Assembler a 32 e a 64 bit

Differenze tra processori a 32 e a 64 bit

- **32 bit**: tutti i registri sono a 32 bit.
- **64 bit**: tutti i registri presentano il prefisso `R` (`RAX`, `RBX`, ...). Ci sono **8 registri in più**: da `R8` a `R15`.
- Anche qui valgono i **suffissi**: permettono di gestire le parti più basse dei registri. Avere più registri comporta un **minore uso della pila**.

Integrazione: nomi dei registri a 64 bit

64 bit	32 bit	16 bit	8 bit
RAX	EAX	AX	AL
RBX	EBX	BX	BL
RCX	ECX	CX	CL
RDX	EDX	DX	DL
RSI	ESI	SI	SIL
RDI	EDI	DI	DIL
RBP	EBP	BP	BPL
RSP	ESP	SP	SPL
R8–R15	R8D–R15D	R8W–R15W	R8B–R15B

Registri di stato

- `RIP`: è come `EIP` nei 32 bit, ma a 64 bit;
- `RFLAGS`: analogo a `EFLAGS`.

Indirizzi in forma canonica

Sono indirizzabili 2^{64} **byte** in teoria; nella pratica se ne usano 2^{48} oppure 2^{57}. I bit più significativi dell'indirizzo (quelli non usati) hanno lo **stesso valore dell'ultimo bit utilizzato** (il 48° oppure il 57°): questi si chiamano **indirizzi in forma canonica**.

Integrazione

È la stessa idea dell'estensione di segno: un indirizzo a 48 bit è esteso a 64 bit ripetendo il suo bit più significativo (il bit 47). Gli indirizzi non in forma canonica provocano un'eccezione.

Modifiche generali all'assembler

- **Suffisso** `Q` (*quad*): operandi a 64 bit, ad esempio `MOVQ`.
- **Indirizzamenti di memoria**:
  - scala: 1, 2, 4, 8;
  - base e indice: **64 bit**;
  - displacement: continua a essere un numero a **32 bit** (**non esiste un displacement a 64 bit**).
- **Non è possibile riferire operandi immediati a 64 bit**: non c'è posto nelle istruzioni. Lo stesso vale per il displacement.

Integrazione

Altre differenze: `PUSH` e `POP` lavorano su 8 byte e usano `RSP`; `CALL` salva in pila un indirizzo di rientro a 8 byte; le istruzioni stringa usano `RSI` e `RDI`. Inoltre, **scrivere un registro a 32 bit azzera i 32 bit alti** del registro a 64 bit corrispondente (ad esempio `MOVL $1, %EAX` azzera la metà alta di `RAX`), mentre scrivere a 8 o 16 bit lascia invariati gli altri bit.

MOVABS

`MOVABS` è un'istruzione che **consente di scrivere come operando sorgente o destinatario un numero a 64 bit**. Va bene sia con l'**indirizzamento immediato** sia con l'**indirizzamento con displacement** (con trasferimento a 8, 16 o 32 bit).

Se usiamo solo `MOV`, l'assemblatore "stampa un insulto" (cioè dà errore) e si blocca.

Integrazione: esempio

```
        MOVQ     $0x1122334455667788, %RAX    # ERRORE: immediato troppo
    grande
        MOVABSQ  $0x1122334455667788, %RAX    # corretto
```

Nella forma con indirizzo a 64 bit (displacement) il dato passa sempre dall'accumulatore (`AL`, `AX`, `EAX` o `RAX`).

Moltiplicazioni, divisioni e altre istruzioni

- Si usa `RCX` al posto di `ECX` (ad esempio come contatore di `LOOP` e `REP`).

- Nella divisione il dividendo è `RDX:RAX`, a **128 bit**.
- Il **limite** nelle rotazioni e nelle traslazioni è **63**: del numero di posizioni si **buttano via i due bit più significativi**, cioè si tengono solo i 6 bit meno significativi.

Integrazione

Il conteggio, di 8 bit (ad esempio in `CL`), viene ridotto modulo 64: a 32 bit si tengono 5 bit (modulo 32). Quindi `SHLQ %CL, %RAX` con `CL` = 65 trasla di 1 posizione.

Parte II

Hardware: le reti combinatorie

Nota

Le dispense su questo argomento **vanno stampate**.

Sistema fisico e modello

In un sistema fisico **vengono prese decisioni**. Il sistema fisico viene modellato attraverso un **modello astratto** che presenta una **variabile logica** W a cui associamo un valore tra 0 e 1 (un bit).

Definizione di rete logica

Una **rete logica** è un **modello astratto** di un sistema fisico costituito da **dispositivi interconnessi**.

- Le informazioni sono **informazioni codificate** e rappresentano **fenomeni fisici distinti**.
- A noi interessa la differenza tra **tensione alta** (1) e **tensione bassa** (0) (nel 95% dei casi).
- Non ci interessa l'*interpretazione* delle variabili, ma le **meccaniche**.

Come si caratterizza

Una rete logica si caratterizza attraverso:

- N **variabili logiche di ingresso**: lo **stato di ingresso** all'istante t è $X(t) = (x_1(t), \ldots, x_N(t))$;
- M **variabili logiche di uscita**: lo **stato di uscita** all'istante t è $Z(t) = (z_1(t), \ldots, z_M(t))$.

Integrazione

Nelle note originali le notazioni erano illeggibili ("$X * X$", "$Z * Z$"): le ho interpretate come $X(t)$ e $Z(t)$, cioè gli stati di ingresso e di uscita all'istante t.

Criteri di classificazione

Dipendenza dallo stato di ingresso (nelle note: "stato di ingresso"):

- **combinatorie**: l'uscita dipende dallo stato di ingresso **attuale**;
- **sequenziali**: l'uscita dipende dalla **storia passata** degli stati di ingresso.

Aggiornamento dello stato di uscita:

- **asincrone**: l'aggiornamento delle uscite è operativo $\forall t$;
- **sincronizzate**: l'aggiornamento avviene in determinati istanti, detti **istanti di sincronizzazione**. Hanno un **clock**.

Da questi criteri si ottengono tre classi:

1. **reti combinatorie**: reti **asincrone e combinatorie**;

2. **reti sequenziali asincrone**;
3. **reti sequenziali sincronizzate**.

Integrazione

La quarta combinazione (combinatoria e sincronizzata) non viene considerata: una rete senza memoria non ha bisogno di un clock.

	Asincrona	**Sincronizzata**
Combinatoria	rete combinatoria	–
Sequenziale	sequenziale asincrona	sequenziale sincronizzata

I limiti del modello

Un **modello** è un modo semplificato di descrivere "qualcosa di complesso": si trascura sempre qualcosa. Dobbiamo quindi **conoscere i limiti** del modello, per non dire cose assurde (cioè per non introdurre errori tramite la modellazione).

Transizione dei segnali

- **Settarsi**: transire a 1.
- **Resettarsi**: transire a 0.

La grandezza fisica modellata dalla variabile logica sarà prima bassa e poi alta, passando per la **fascia di indeterminazione** durante la crescita (e può anche tornare indietro).

Non si conosce l'**istante esatto in cui avviene un cambiamento**, ma si sa che avviene in un intervallo $\Delta t \ll \Delta T$. Non è un problema.

Integrazione

Qui Δt è la durata della transizione e ΔT è il tempo che passa tra due cambiamenti successivi dello stato di ingresso: la transizione è molto più breve, quindi la possiamo trattare come istantanea.

Non contemporaneità

La **non contemporaneità** è un problema perché **non rappresenta la nostra interpretazione della rete**. Nel sistema reale la **contemporaneità fisica non esiste**: una variabile cambierà sempre prima di un'altra. Ci si aspetta che la rete reagisca in un modo o nell'altro, ma **non abbiamo controllo di ciò che accade "nel mezzo"**.

- Nelle reti **sequenziali** diventa un problema **dal momento in cui succede in poi** (lo stato sbagliato può restare memorizzato).
- Se la rete è **sincronizzata** (e ci si trova lontano dagli istanti in cui la rete "si sveglia") **non è assolutamente un problema**.

Regola

Stati di ingresso consecutivi devono essere adiacenti, cioè differire solo di un bit l'uno dall'altro.

Integrazione

Esempio: passando dall'ingresso `01` a `10` due bit cambiano "insieme"; in realtà uno cambia prima dell'altro e la rete vede per un breve istante `00` oppure `11`, producendo un'uscita spuria. In una rete combinatoria è un transitorio innocuo se si aspetta che vada a regime; in una sequenziale può essere memorizzato.

Le reti combinatorie

Caratterizzazione (mancante nelle note)

Una rete combinatoria ha N ingressi e M uscite e **non ha memoria**: lo stato di uscita all'istante t dipende solo dallo stato di ingresso allo stesso istante,

$$Z(t) = F\big(X(t)\big), \qquad F : \{0,1\}^N \to \{0,1\}^M.$$

Ogni rete combinatoria è rappresentata da un **simbolo** e la usiamo come componente.

Tempo di accesso

Il **tempo di accesso** (o di attraversamento, o di risposta) è **piccolo ma non nullo** (ci sono effetti fisici che fanno perdere tempo). È scritto sulle "scatole" delle porte logiche e rappresenta un **tempo massimo**. Se non lo si attende, **succedono cose strane**.

Regola

Gli ingressi si cambiano quando la rete è andata a regime e soprattutto **un bit alla volta**.

Descrizione di una rete combinatoria

La descrizione comprende:

- il **numero di ingressi**;
- il **numero di uscite**;
- una **descrizione funzionale**.

La si può dare in diversi modi:

- **a parole**;
- con **linguaggi** come *Verilog*;
- con una **tabella di verità**: si elencano a sinistra tutti gli stati di ingresso, a destra lo stato di uscita $F(\mathrm{IN}_1, \ldots, \mathrm{IN}_K)$.

Il simbolo "-" (don't care)

Nelle uscite possiamo trovare anche "-": significa **non specificato**, cioè *don't care*. Non significa che l'uscita sia nella fase di indeterminazione: significa che **non ci interessa se l'uscita è 0 o 1**.

E allora perché le consideriamo? Perché sono libertà che possiamo sfruttare nella **sintesi**: potendo scegliere 0 o 1, si sceglie ciò che permette un **risparmio di porte**.

Descrizione e sintesi

- La **descrizione** è il modo formale per specificare **cosa faccia** la rete (relazione ingresso–uscita).
- La **sintesi** è il **progetto di realizzazione**: consiste nel **limitare il numero di porte**, perché "costano, si rompono…".

Quindi dobbiamo fare **prima** la **descrizione** e **poi** la **sintesi**.

Integrazione

Nelle note questo passaggio era confuso: ho unito le frasi sui *don't care* e sulla sintesi, intendendo che le uscite non specificate servono proprio a semplificare la sintesi.

Reti con più uscite

Importante

Non ha senso trattare reti combinatorie con più uscite, perché le si può pensare come un **insieme di reti combinatorie** che hanno tutti gli ingressi e **una sola uscita**. Trattiamo quindi **solo reti con una sola uscita**.

Reti combinatorie per numero di ingressi

Con n ingressi la tabella di verità ha 2^n righe, e il numero di reti diverse è 2^{2^n}.

Zero ingressi (caso degenere)

Per generare **1** si attacca un filo all'**alimentazione**; altrimenti, per generare **0**, a **terra** (massa).

Un ingresso

Con $n = 1$ ci sono $2^{2^1} = 4$ reti:

- **Invertitore** (NOT): "il pallino" sul simbolo significa *inverti*.
- **Elemento neutro** o **buffer**: serve a "perdere tempo" (introdurre un ritardo) o a **rigenerare i segnali elettrici**, nel caso in cui x sia degradato da effetti indesiderati su un conduttore (ad esempio il **rumore**).
- **Casi degeneri** (vedere dispense): Z collegata all'alimentazione ($Z = 1$) oppure a massa ($Z = 0$). È un filo, non una rete.

Nota

Quando una rete logica funziona, la tensione viene dall'**uscita** (della porta) e non dall'alimentazione. *(Frase poco chiara nelle note: il senso è che la porta "ricostruisce" il segnale, come fa il buffer; verificare sulle dispense.)*

Due ingressi

Sono $2^{2^2} =$ **16 reti combinatorie diverse**; si marcano quelle con **nomi interessanti** (per vederle tutte, vedere le dispense).

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

Integrazione

Delle 16 reti, 6 sono quelle sopra; le altre 10 sono: costante 0, costante 1, A, B, $\overline{A}$, $\overline{B}$, $A\overline{B}$, $\overline{A}B$, $A+\overline{B}$, $\overline{A}+B$. `XOR` vale 1 quando gli ingressi sono *diversi*; `XNOR` è il suo complemento (vale 1 quando sono uguali).

Nota

Si scrive **XNOR**, *non* NXOR.

- **Qualunque cosa può essere fatta a forza di porte fatte così.**
- È possibile **ampliare** `AND` e `OR` usando più ingressi: la `AND` a N ingressi vale 1 se e solo se *tutti* gli ingressi valgono 1; la `OR` vale 1 se *almeno uno* vale 1.

Integrazione

Un insieme di porte con cui si può realizzare qualunque rete combinatoria si dice *completo*: ad esempio {AND, OR, NOT}. Anche la sola `NAND` (o la sola `NOR`) è completa.

Consigli

Fare **sempre** la tabella di verità! Fare esercizi.

— Fine Lezione 7 —

Questa lezione, come le sue precedenti e successive, può contenere errori e/o imprecisioni. Per il bene di tutti, è buona norma far notare all'autore l'errore o l'imprecisione così da risolvere nel minor tempo possibile. I canali di comunicazione, per eventuali avvisi generali o di errori trovati negli appunti, sono in cima al file. Grazie per la collaborazione e Buono studio!

Lezione 8 | 2 ottobre 2026

Il sottoprogramma `outline`, Compilazione ed esecuzione, Il debugger

All'esame

Le versioni **PDF delle esercitazioni** sono portabili all'esame: **solo quelle**, e solo se **non contengono esercizi svolti**.

Il sottoprogramma `outline`

`outline` prende un **indirizzo** v **in** `%EBX` e stampa i caratteri a partire da quell'indirizzo **finché non incontra un ritorno carrello** `"\r"` (CR, codice `0x0D`). Useremo questo terminatore **per convenzione** e perché è lo stesso dell'esame.

Integrazione

Questo chiarisce le note della lezione del 30 settembre su `inline`/`outline`: il messaggio da stampare deve quindi **terminare con il carattere CR**, non con lo zero finale di `.asciz`.

```
msg:    .ascii  "Ciao\r"

        LEAL    msg, %EBX       # EBX = indirizzo del messaggio
        CALL    outline         # stampa fino al ritorno carrello
```

Compilazione ed esecuzione

- **File sorgente**: hanno estensione `.s` e contengono il codice.
- **File eseguibili**: servono a **eseguire il codice assemblato**.

Per ottenere l'eseguibile si scrive:

```
./assemble sorgente
```

Il debugger

Il **debugger** serve a **seguire e monitorare** il nostro programma, per vedere come funziona e per capire **perché non ha il comportamento che ci aspettiamo**, se qualcosa va storto. Permette di verificare lo **stato dei registri** e della **memoria**.

Si chiama con:

```
./debug.ps1 eseguibile
```

Breakpoint

Bisogna **mettere dei breakpoint**: altrimenti il codice eseguirebbe tutto il programma senza fermarsi.

Integrazione

Si usa il comando `break` (abbreviato `b`), seguito da un'etichetta o da un indirizzo: ad esempio `break main`. Il programma parte con `run` (`r`) e si ferma al breakpoint. Per questo, come visto nella lezione del 29 settembre, conviene che la prima istruzione sia una `NOP`: ci si può fermare lì. Con `continue` (`c`) l'esecuzione riprende fino al breakpoint successivo.

Comandi principali

Comando	Effetto
`step`	fa passare all'**istruzione successiva**
`info registers`	stampa lo **stato di tutti i registri**
`info registers` *nomi*	stampa **solo i registri indicati** (ad esempio `info registers ebx`)
`x/NFU` *indirizzo*	**esamina la memoria** (*examine memory*)

Integrazione

I comandi sono quelli di GDB. Per programmi scritti in assembler, l'avanzamento *istruzione per istruzione* si ottiene di solito con `stepi` (`si`), perché `step` ragiona per righe di un linguaggio ad alto livello; conviene provare quale dei due si comporta come atteso nell'ambiente dell'esercitazione.

Il comando `x`

`x` sta per *examine memory*. Si scrive `x/NFU` seguito dall'indirizzo, dove:

- **N**: quanti elementi mostrare (un numero);
- **F**: il **formato di interpretazione**;
- **U**: la **dimensione dell'elemento** (8, 16 o 32 bit).

Integrazione

Formato (F)		Dimensione (U)	
`x`	esadecimale	`b`	byte (8 bit)
`d`	decimale con segno	`h`	halfword (16 bit)
`u`	decimale senza segno	`w`	word (32 bit)
`t`	binario	`g`	giant (64 bit)
`c`	carattere		
`s`	stringa		

Esempio

Nell'esempio dell'esercitazione, `%EBX` contiene l'**indirizzo della prima locazione del messaggio**, e si usa `NFU` = `1cb`: cioè **1** elemento, formato **c** (carattere), dimensione **b** (byte).

Integrazione: sessione tipo

```
./assemble esempio
./debug.ps1 esempio
(gdb) break main
(gdb) run
(gdb) stepi
(gdb) info registers
(gdb) info registers ebx
(gdb) x/1cb $ebx          # primo carattere del messaggio
(gdb) x/8cb $ebx          # primi 8 caratteri
(gdb) x/s $ebx            # messaggio come stringa
```

Nei comandi di GDB i registri si scrivono con il prefisso `$` (`$ebx`), non con `%`.

— Fine Lezione 8 —